

Program Design Discussion

This program is designed using an object-oriented approach to keep responsibilities separated and make the code easier to test, read, and extend. The overall flow is controlled by a Game class, which manages player setup, turn-taking, win/draw detection, and statistics across multiple games. A Board class stores and displays the TicTacToe grid and enforces legal moves. A Player class stores each player's identity (first/last name), chosen piece (x or o), and their win/loss/draw totals. The main function is intentionally small: it creates a Game object and calls setup() and run() to start execution.

Input validation is handled carefully to prevent crashes and bad state. Player names are validated to ensure exactly two alphabetic tokens (first and last name). Move input is validated to ensure it is a single integer with no extra tokens and that the chosen board position is within 1–9 and currently empty. The game loop repeats until a win or draw occurs, updates statistics, prints results, and then asks whether the players want to play again.

Data Structure Design Discussion

Player Data Structure

Each Player object encapsulates:

- string firstName, string lastName: stores identity for prompts and reporting.
- char piece: stores whether the player is 'x' or 'o'.
- int wins, int losses, int draws: stores cumulative stats across games.

This design makes player state persistent between games while keeping it isolated from board logic. Using strings supports flexible name input and output formatting. Integers are efficient for tracking results.

Board Data Structure

The game board is represented using:

- char cells[9]: a fixed-size array storing the 9 board positions.
- int moveCount: tracks how many moves have been made.
- bool winningPositions[9]: reserved for tracking winners (even though the current code marks winners by uppercasing the board cells).

A fixed-size array is the most direct and efficient structure for TicTacToe because the board size never changes (always 9 positions). Array indexing is $O(1)$, so checking and updating a cell is fast. The program also uses a custom mapping between the user's

“position numbers” (1–9) and internal indices. This mapping allows the displayed board to match the specific numbering pattern used in `display()`.

Game Control Structure

The Game class acts as the coordinator and contains:

- Two Player objects: `player1`, `player2`
- One Board object: `board`
- Integers for game state tracking: `currentPlayerIndex`, `totalGamesPlayed`, `firstGameStarterIndex`, `lastGameResult`

This structure makes the game state easy to manage because all coordination variables live in one place. The `currentPlayerIndex` is used to select the active player without duplicating code for turns. `totalGamesPlayed` supports reporting, and `lastGameResult` supports deciding who starts the next game.

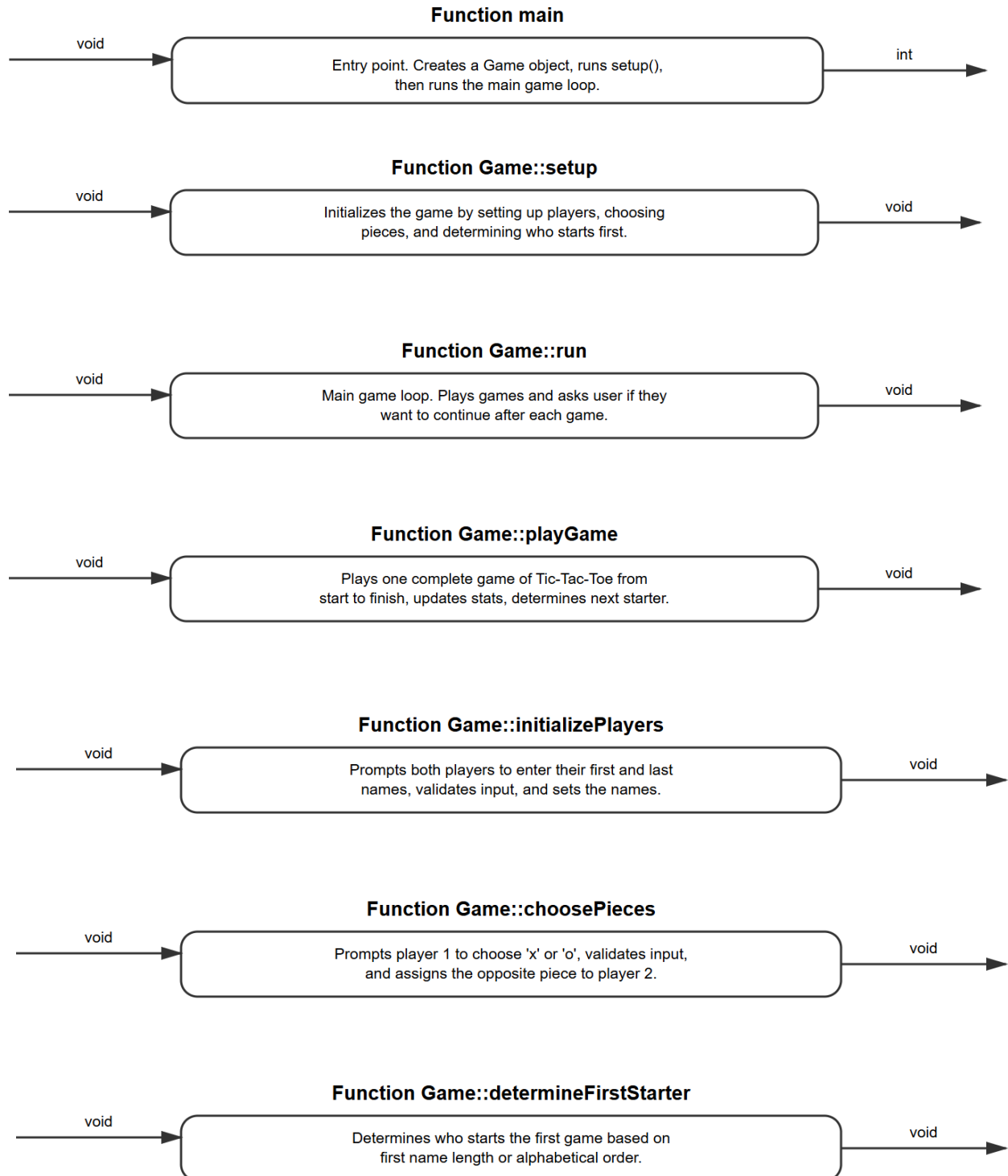
Efficiency and Complexity Notes

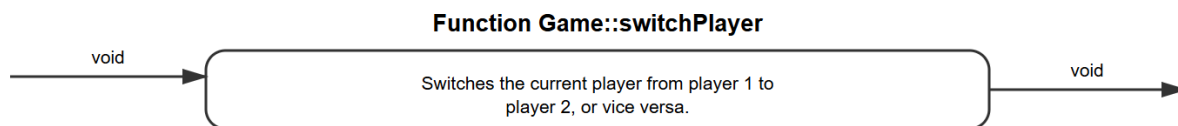
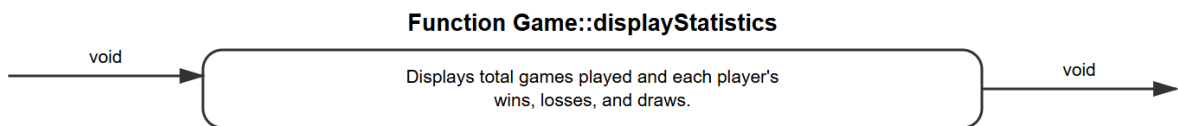
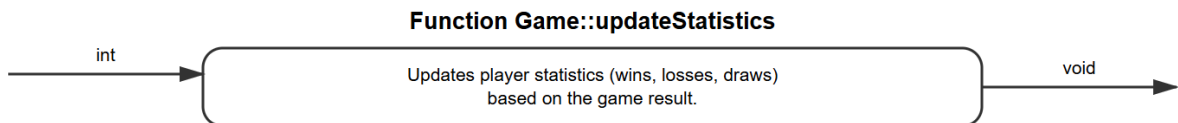
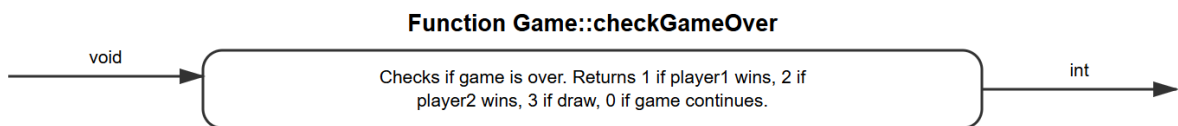
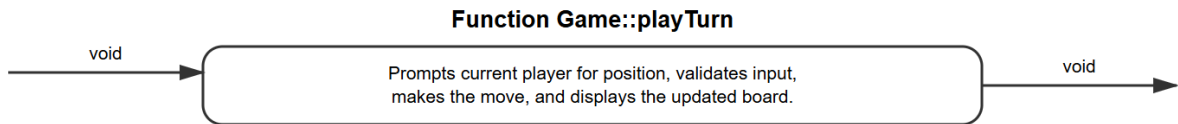
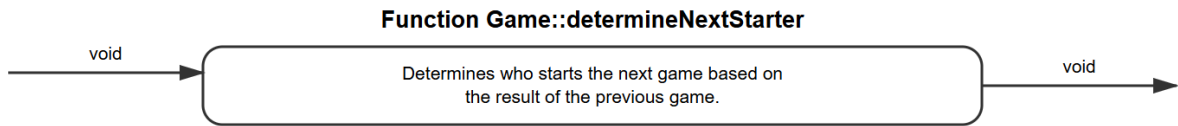
- Making a move is $O(1)$ (array access).
- Checking for a win is $O(1)$ because it tests a constant number of winning patterns (8).
- Checking if the board is full is $O(9)$, which is effectively constant time for this problem size.
- Memory usage is constant because all data structures have fixed size (two players + 9 board cells).

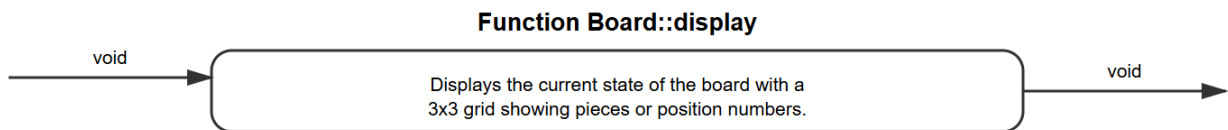
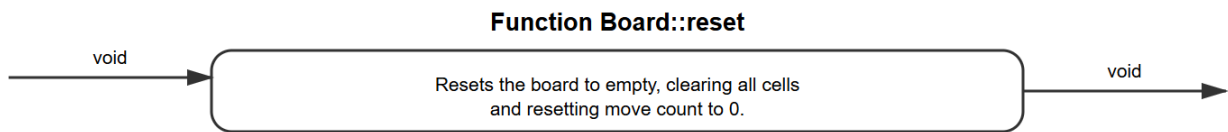
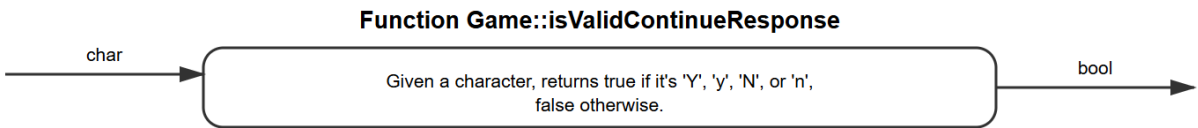
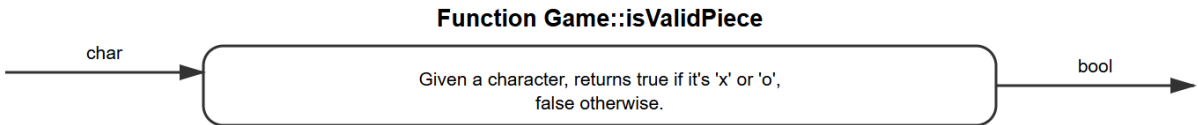
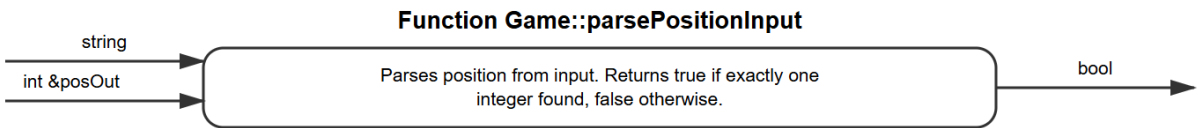
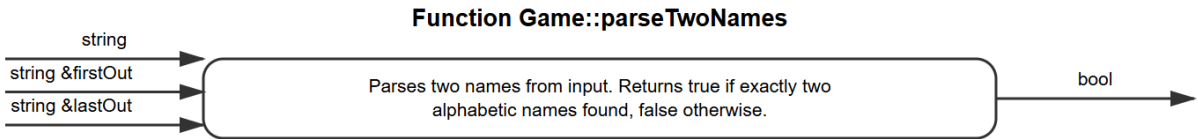
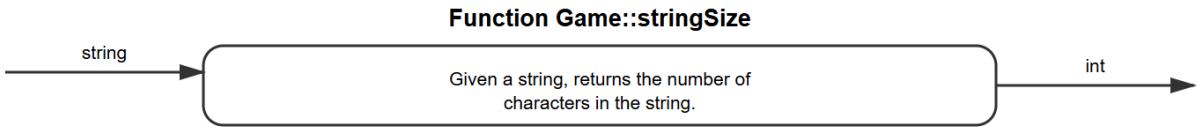
Hierarchy Chart:

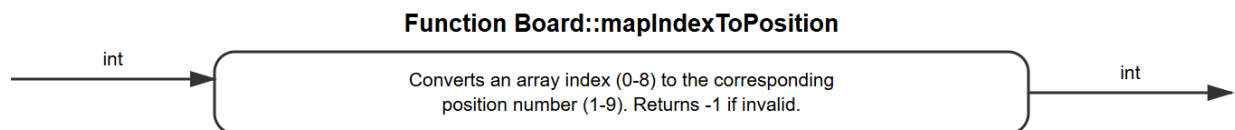
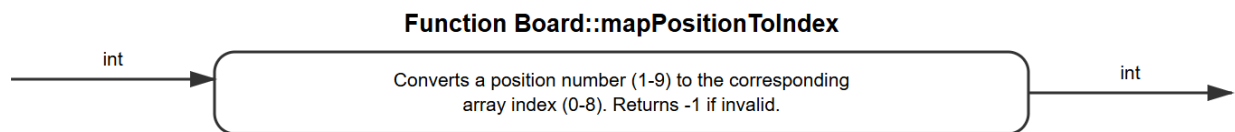
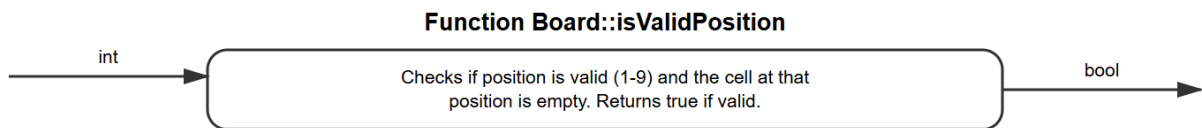
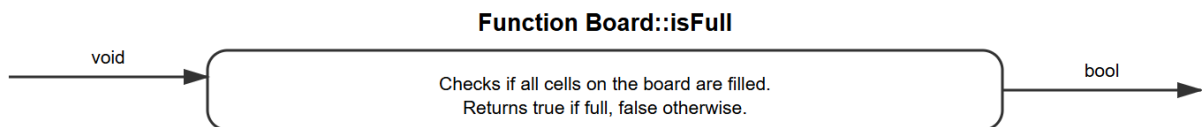
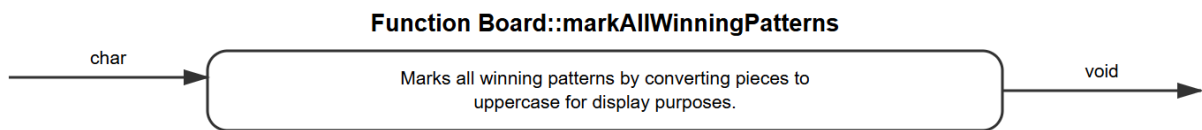
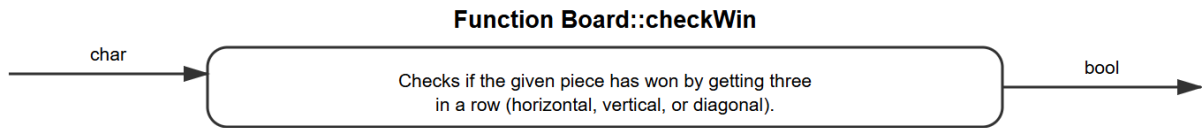
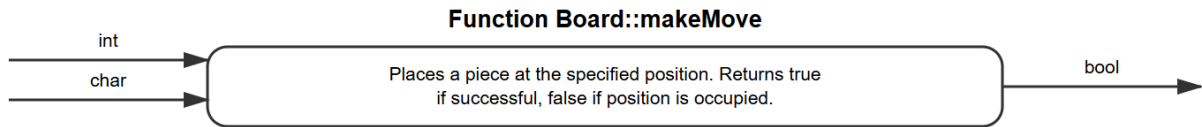


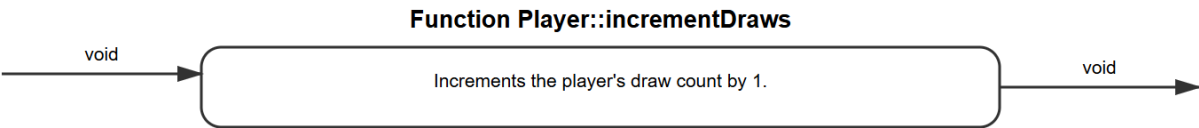
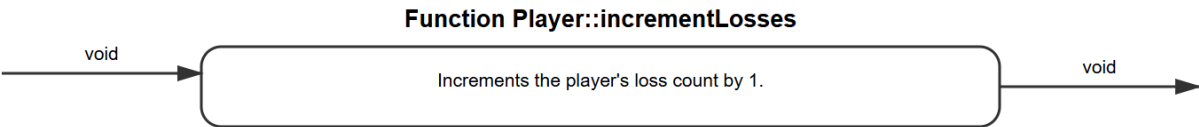
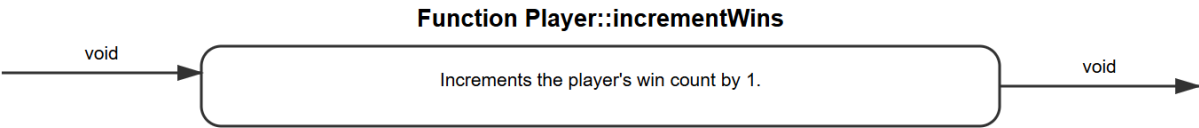
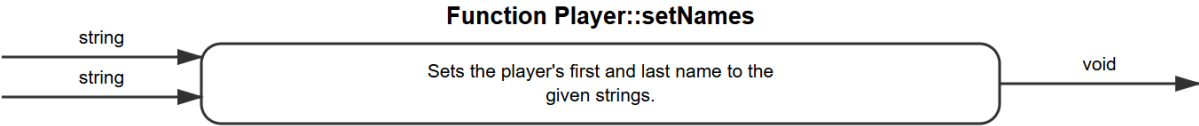
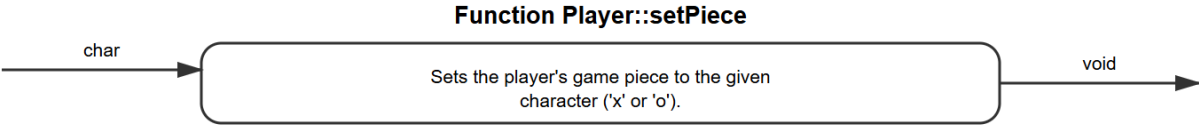
Procedure Specification











Test Case 1 — Valid Setup

Inputs: John Doe, Amy Smith, piece x

Justification: Confirms normal setup, name parsing, piece assignment, starter logic.

Expected Output: Prompts for names and piece; prints assignments (John will be: x, Amy will be: o); starter printed based on first-name length rule.

Test Case 2 — Invalid Name (Non-Letters)

Inputs: Br3nt Foxworth → Brent Foxworth; Ava Brown; piece o

Justification: Ensures parseTwoNames() rejects non-alphabetic input.

Expected Output: Invalid name message + reprompt; continues after valid input.

Test Case 3 — Invalid Name (Extra Tokens)

Inputs: John A Doe → John Doe; Amy Smith

Justification: Confirms exactly two names required.

Expected Output: Invalid name message + reprompt.

Test Case 4 — Invalid Piece (Wrong Length)

Inputs: x0 → x

Justification: Ensures piece must be exactly one character.

Expected Output: “Invalid piece! Please enter exactly one character...”

Test Case 5 — Invalid Piece (Wrong Character)

Inputs: z → o

Justification: Confirms only x or o allowed.

Expected Output: “Invalid piece! Please enter 'x' or 'o' only.”

Test Case 6 — Invalid Move (Non-Numeric)

Inputs: five → 1

Justification: Validates numeric-only position input.

Expected Output: "Invalid input! Please enter a single number between 1 and 9." then board updates after 1.

Test Case 7 — Invalid Move (Extra Tokens)

Inputs: 1 2 → 1

Justification: Confirms single-number enforcement.

Expected Output: Same invalid input message; board updates after valid input.

Test Case 8 — Out-of-Range Move

Inputs: 0, 10 → 5

Justification: Ensures 1–9 range validation.

Expected Output: "Invalid position! Please enter a number between 1 and 9." then board updates after 5.

Test Case 9 — Taken Position

Inputs: P1: 1; P2: 1 → 2

Justification: Confirms board rejects occupied cell.

Expected Output: "That position is already taken!" then board updates after 2.

Test Case 10 — Player 1 Win (Row)

Moves: 1,6,2,7,3

Justification: Tests horizontal win detection.

Expected Output: <Player1 Full Name> wins! and board redisplayed with winning marks uppercased.

Test Case 11 — Player 2 Win (Diagonal)

Moves: 1,5,6,9,2,4

Justification: Tests diagonal win detection.

Expected Output: <Player2 Full Name> wins! and board redisplayed.

Test Case 12 — Draw

Moves: 1,2,3,5,4,6,8,7,9

Justification: Tests full-board draw detection.

Expected Output: "It's a draw!" and both draw counts increment.

Test Case 13 — Invalid Continue (Length)

Inputs: Yes → Y

Justification: Continue input must be one character.

Expected Output: "Invalid input! Please enter exactly one character..." then new game starts.

Test Case 14 — Invalid Continue (Character)

Inputs: Q → N

Justification: Only Y/y/N/n accepted.

Expected Output: "Invalid input! Please enter 'Y' for yes or 'N' for no." then "Thanks for playing!"

Test case 1:

```
Player 1 please enter your first and last name: John Doe
Player 2 please enter your first and last name: Amy Smith
John, please enter your piece (x or o): x
John will be: x
Amy will be: o
Amy will start
---- Welcome to TicTacToe, please follow each prompt to play ----
Total games played: 1
+---+---+---+
| 1 | 6 | 2 |
+---+---+---+
| 7 | 5 | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+
Amy, please select your position: |
```

Test case 2:

```
Player 1 please enter your first and last name: Br3nt Foxworth
Invalid name! Name must contain only letters with exactly one space between first and last name.
Please try again.
Player 1 please enter your first and last name: Brent Foxworth
Player 2 please enter your first and last name: Ava Brown
Brent, please enter your piece (x or o): o
Brent will be: o
Ava will be: x
Ava will start
---- Welcome to TicTacToe, please follow each prompt to play ----
Total games played: 1
+---+---+---+
| 1 | 6 | 2 |
+---+---+---+
| 7 | 5 | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+
Ava, please select your position: |
```

Test case 3:

```
Player 1 please enter your first and last name: John A Doe
Invalid name! Name must contain only letters with exactly one space between first and last name.
Please try again.

Player 1 please enter your first and last name: John Doe

Player 2 please enter your first and last name: |
```

Test case 4:

```
John, please enter your piece (x or o): xo
Invalid piece! Please enter exactly one character ('x' or 'o').

John, please enter your piece (x or o): x

John will be: x
Brent will be: o

John will start
---- Welcome to TicTacToe, please follow each prompt to play ----

Total games played: 1

+---+---+---+
| 1 | 6 | 2 |
+---+---+---+
| 7 | 5 | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

John, please select your position: |
```

Test case 5:

```
John, please enter your piece (x or o): z
Invalid piece! Please enter 'x' or 'o' only.

John, please enter your piece (x or o): o

John will be: o
Brent will be: x

John will start
---- Welcome to TicTacToe, please follow each prompt to play ----

Total games played: 1

+---+---+---+
| 1 | 6 | 2 |
+---+---+---+
| 7 | 5 | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

John, please select your position: |
```

Test case 6:

```
---- Welcome to TicTacToe, please follow each prompt to play ----  
  
Total games played: 1  
  
+---+---+---+  
| 1 | 6 | 2 |  
+---+---+---+  
| 7 | 5 | 8 |  
+---+---+---+  
| 3 | 9 | 4 |  
+---+---+---+  
  
brent, please select your position: five  
Invalid input! Please enter a single number between 1 and 9.  
  
brent, please select your position: 5  
  
+---+---+---+  
| 1 | 6 | 2 |  
+---+---+---+  
| 7 | x | 8 |  
+---+---+---+  
| 3 | 9 | 4 |  
+---+---+---+  
  
john, please select your position: |
```

Test case 7:

```
+---+---+---+  
| 1 | 6 | 2 |  
+---+---+---+  
| 7 | x | 8 |  
+---+---+---+  
| 3 | 9 | 4 |  
+---+---+---+  
  
john, please select your position: 1 2  
Invalid input! Please enter a single number between 1 and 9.  
  
john, please select your position: 1  
  
+---+---+---+  
| o | 6 | 2 |  
+---+---+---+  
| 7 | x | 8 |  
+---+---+---+  
| 3 | 9 | 4 |  
+---+---+---+  
  
brent, please select your position: |
```

Test case 8:

```
+---+---+---+
| o | 6 | 2 |
+---+---+---+
| 7 | x | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

brent, please select your position: 0
Invalid position! Please enter a number between 1 and 9.

brent, please select your position: 10
Invalid position! Please enter a number between 1 and 9.

brent, please select your position: 6

+---+---+---+
| o | x | 2 |
+---+---+---+
| 7 | x | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

john, please select your position: |
```

Test case 9:

```
+---+---+---+
| o | x | 2 |
+---+---+---+
| 7 | x | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

john, please select your position: 1
That position is already taken! Please choose another position.

john, please select your position: 3

+---+---+---+
| o | x | 2 |
+---+---+---+
| 7 | x | 8 |
+---+---+---+
| o | 9 | 4 |
+---+---+---+

brent, please select your position: |
```

Test case 10:

```
brent f wins!

+---+---+---+
| X | X | X |
+---+---+---+
| o | o | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

Total Games Played: 1

brent f's stats:
Wins: 1
Losses: 0
Draws: 0

lauren f's stats:
Wins: 0
Losses: 1
Draws: 0

lauren will start the next game

Would you like to play another game? (Y/N):
```

Test case 11:

```
lauren f wins!

+---+---+---+
| 0 | x | x |
+---+---+---+
| 7 | 0 | 8 |
+---+---+---+
| 3 | 9 | 0 |
+---+---+---+

Total Games Played: 2

brent f's stats:
Wins: 1
Losses: 1
Draws: 0

lauren f's stats:
Wins: 1
Losses: 1
Draws: 0

brent will start the next game

Would you like to play another game? (Y/N): |
```


Test case 12:

```
+---+---+---+
| x | o | x |
+---+---+---+
| x | o | o |
+---+---+---+
| o | x | x |
+---+---+---+

It's a draw!
Total Games Played: 5

brent f's stats:
Wins: 2
Losses: 2
Draws: 1

lauren f's stats:
Wins: 2
Losses: 2
Draws: 1

lauren will start the next game

Would you like to play another game? (Y/N): |
```

Test case 13:

```
Would you like to play another game? (Y/N): yes
Invalid input! Please enter exactly one character ('Y' or 'N').

Would you like to play another game? (Y/N): y

Total games played: 6

+---+---+---+
| 1 | 6 | 2 |
+---+---+---+
| 7 | 5 | 8 |
+---+---+---+
| 3 | 9 | 4 |
+---+---+---+

lauren, please select your position: |
```

Test case 14:

```
Would you like to play another game? (Y/N): 8
Invalid input! Please enter 'Y' for yes or 'N' for no.

Would you like to play another game? (Y/N): q
Invalid input! Please enter 'Y' for yes or 'N' for no.

Would you like to play another game? (Y/N): n

Thanks for playing!
```

Limitations and Future Works:

The current version of the program is limited to two human players and only runs in the console, meaning it does not include a computer opponent or a graphical user interface. The board size is fixed at 3×3, so it cannot support larger grids or custom game variations, and player statistics are not saved once the program ends. Additionally, the game logic is closely tied to console input and output, which makes it harder to expand or reuse in other formats. In the future, the program could be improved by adding an AI opponent, creating a graphical interface, supporting dynamic board sizes, saving player statistics to a file, and further separating the game logic from the user interface to make the system more scalable and maintainable.